Now let's tackle the main training loop, shown in listing 8.12. We initialize the first state using the `prepare_initial_state(...)` function we defined earlier, which just takes the first frame and repeats it three times along the channel dimension. We also set up a `deque` instance, to which we will append each frame as we observe them. The `deque` is set to a `maxlen` of `3`, so only the most recent three frames are stored. We convert the `deque` first to a list and then to a PyTorch tensor of dimensions $1 \times 3 \times 42 \times 42$ before passing it to the Q-network.

Listing 8.12. The training loop

```
epochs = 3500
env.reset()
state1 = prepare_initial_state(env.render('rgb_array'))
eps=0.15
losses = []
episode_length = 0
switch_to_eps_greedy = 1000
state_deque = deque(maxlen=params['frames_per_state'])
e_reward = 0.
last_x_pos = env.env.env._x_position                                    1
ep_lengths = []
use_explicit = False
for i in range(epochs):
    opt.zero_grad()
    episode_length += 1
    q_val_pred = Qmodel(state1)                                         2
    if i > switch_to_eps_greedy:                                        3
        action = int(policy(q_val_pred,eps))
    else:
        action = int(policy(q_val_pred))
    for j in range(params['action_repeats']):                           4
        state2, e_reward_, done, info = env.step(action)
        last_x_pos = info['x_pos']
        if done:
            state1 = reset_env()
            break
        e_reward += e_reward_
        state_deque.append(prepare_state(state2))
    state2 = torch.stack(list(state_deque),dim=1)                       5
    replay.add_memory(state1, action, e_reward, state2)                 6
    e_reward = 0
    if episode_length > params['max_episode_len']:                      7
        if (info['x_pos'] - last_x_pos) < params['min_progress']:
            done = True
        else:
            last_x_pos = info['x_pos']
```